AlphaMirror — Project Documentation

Verified Smart-Money Copy-Trading on AVE Cloud Skill

Team **Percayalah**
Leader: Pugar Huda Mantoro

April 15, 2026

---

1. Executive Summary

AlphaMirror is a non-custodial, verified copy-trading agent built on AVE Cloud Skill for the **AVE Claw Hackathon 2026** (Complete Application track). It turns AVE's built-in smart-money classifier into a ranked, audit-trailed list of wallets that retail users can actually trust.

Most copy-trading platforms rank wallets by raw profit. This misleads retail: one lucky 100x trade dominates the leaderboard, and dormant whales look like active alpha hunters. **AlphaMirror cross-checks AVE's `smart_wallets` classification against six independent quality signals** and produces an APPROVED / REVIEW / REJECTED verdict with a 0–100 score and full human-readable breakdown.

The system uses **11 endpoints across 2 AVE skills** entirely on the **free API tier** (no whitelist, no proxy wallet, no pro plan required), runs at **1 TPS with a thread-safe client-side rate limiter**, and ships with an **offline demo mode** backed by fixture data for reliable video recording. The entire pipeline is self-custody: we never touch user keys. Trades are composed as unsigned AVE `trade-chain-wallet quote` commands that the user signs in their own wallet.

On a live run against BSC on 2026-04-15 with 10 candidates:

- 2 APPROVED (top scores 92 and 78)
- 7 REVIEW (mid-tier requiring human judgment)
- **1 REJECTED**: wallet `0xacf1e…` — flagged by AVE as smart money for 433 days, but our verification shows a total loss of $2,344

This single rejected wallet is the entire product value in one data point: **not every wallet AVE labels as "smart money" is actually worth mirroring, and AlphaMirror proves which ones are.**

2. Problem Statement

Retail copy-trading platforms (eToro, Binance Copy Trading, GMX, dozens of Telegram bots) rank wallets by top-line P&L. This creates three systemic failure modes:

1. **One-hit wonders dominate.** A wallet with one lucky 100x trade and nine break-even trades shows identical "total profit" to a wallet with twenty consistent 2–5x wins. Retail cannot distinguish them from the leaderboard display alone.
2. **Dormant smart money gets followed.** Wallets that were genuinely profitable years ago but haven't traded in 90 days still appear at the top because the tools don't weight recency.
3. **Unprofitable "smart money" exists.** Even AVE's own `smart_wallets` endpoint — which uses volume, activity, and historical behavior for classification — returns some wallets that are **net negative** when the numbers are checked directly. AVE's classifier gives you a candidate pool; it does not give you a verdict.

The existing tools that try to solve this (Nansen, Arkham, DeBank's Smart Wallet list) are either closed-source enterprise products or do not expose the per-wallet P&L and profit-tier distribution that AVE provides for free. There is a whitespace for a public, transparent, auditable verification layer — and AlphaMirror fills it.

3. Solution Overview

AlphaMirror runs a 5-phase pipeline on top of AVE Cloud's `data-rest` and `trade-chain-wallet` skills:

Phase	Step	AVE Skill Endpoint(s)
1	**Discover** — pull candidate smart wallets from AVE's classifier	`smart_wallets`
2	**Verify** — cross-check each candidate with independent quality signals	`wallet_info`, `wallet_tokens`
3	**Score** — compute 0–100 Verification Score + verdict	(offline logic)
4	**Monitor** — poll approved wallets for new positions + risk gate	`wallet_tokens`, `risk`
5	**Mirror** — build self-custody trade quote for user to sign	`trade-chain-wallet quote`

3.1 Phase 1 — Discovery

A single call to `/address/smart_wallet/list?chain=bsc` returns AVE's top-100 smart-money candidates for the chain, each enriched with: `total_profit`, `total_profit_rate`, `token_profit_rate` (win rate), `total_volume`, `total_trades`, `buy_trades`, `sell_trades`, `last_trade_time`, `tag_items` (top traded tokens), and a per-tier profit distribution histogram with 8 buckets (10x+, 5–10x, 3–5x, 2–4x, 1.1–2x, flat, -10..-50%, -50..-100%).

This endpoint alone provides far more data than any public alternative, and is what makes AlphaMirror's deep verification possible on the free tier — we don't need to reconstruct P&L from raw transaction history.

3.2 Phase 2 — Verification

For each candidate, two additional API calls:

- `wallet_info` — returns `wallet_age` (Unix timestamp of first activity), total balance, win ratio, and per-chain aggregates.
- `wallet_tokens` — returns current token holdings with `value_usd`, amounts, and blue-chip classification.

These calls let us cross-reference AVE's historical claim with the current state of the wallet. A wallet AVE flags as profitable should still have a balanced portfolio — if it has a $0 portfolio after claiming $500k profit, either the data is wrong or the wallet has fully rotated out and is no longer "active smart money."

3.3 Phase 3 — Scoring

The 6-signal Verification Score (detailed in Section 5 below) produces:

- **APPROVED** (≥70): mirror-worthy, passes all stress tests
- **REVIEW** (40–69): show the user, let them judge
- **REJECTED** (<40): AVE flagged it, we don't agree — stay away

3.4 Phase 4 — Monitor

For each approved wallet the user chooses to track, we poll `wallet_tokens` every 60 seconds (configurable, free-tier safe) and compute the diff. New token addresses = open signal; disappeared tokens = exit signal. Each new position triggers a `risk` call (`/contracts/{address}-{chain}`) to check for honeypot, high tax, or owner-abusable permissions before we surface it to the user.

3.5 Phase 5 — Mirror

When the user wants to mirror a position, AlphaMirror composes the exact AVE `trade-chain-wallet quote` shell command, pre-filled with:

- Chain
- Input token: the default stablecoin for that chain (e.g. USDT on BSC)
- Output token: the target token

- Input amount: converted to the token's smallest unit (wei-style)
- Swap type: `buy`

The command is displayed in the UI. The user copies it, runs it in their own shell, and signs the resulting unsigned transaction in their own wallet. **AlphaMirror never touches private keys.**

4. AVE Cloud Skill Integration

AlphaMirror uses **11 endpoints across 2 AVE skills**, entirely on the **free API tier**. No whitelist, no proxy wallet, no Pro plan.

4.1 `data-rest` skill

Endpoint	Path	Phase	Purpose
`smart_wallets`	`GET /address/smart_wallet/list`	1	Candidate pool — AVE's built-in smart-money classifier with profit, win rate, profit-tier distribution
`wallet_info`	`GET /address/walletinfo`	2	Wallet age (anti-sybil), total balance, per-chain aggregates
`wallet_tokens`	`GET /address/walletinfo/tokens`	2, 4	Current portfolio snapshot + polling diff
`address_pnl`	`GET /address/pnl`	reserved	Per-token P&L drill-down (for future deep-dive UI)
`search`	`GET /tokens`	discovery	Token resolution by symbol
`token`	`GET /tokens/{addr}-{chain}`	enrichment	Price / liquidity / volume lookup
`kline-token`	`GET /klines/token/{addr}-{chain}`	enrichment	OHLCV price context
`risk`	`GET /contracts/{addr}-{chain}`	4	Honeypot / tax / owner-permissions safety gate
`trending`	`GET /tokens/trending`	discovery	Optional discovery context
`address_txs`	`GET /address/tx`	reserved	Deeper wallet forensics

4.2 `trade-chain-wallet` skill

Endpoint	Path	Phase	Purpose
`quote`	(via `ave_trade_rest.py quote`)	5	Build self-custody mirror trade preview; user signs separately

4.3 Why we chose the skills we did

- **`data-rest` over `data-wss`**: Free tier blocks `data-wss` (WebSocket requires `API_PLAN=pro`). AlphaMirror's polling-based monitor is a deliberate trade-off: we lose sub-second reactivity but we don't need a Pro plan, which keeps AlphaMirror accessible to every hackathon attendee.
- **`trade-chain-wallet` over `trade-proxy-wallet`**: Proxy wallet requires `API_PLAN=normal` minimum and asks the user to deposit funds into an AVE-managed wallet. Chain wallet is self-custody — the user's keys never leave their wallet app. This is a deliberate positioning choice that aligns with Web3 values and keeps AlphaMirror free-tier compatible simultaneously.

5. Verification Methodology

The Verification Score (0–100) is a weighted combination of six independent signals. Weights were tuned to emphasize **profitability and consistency** over portfolio size — a $500k stagnant blue-chip wallet is less valuable to mirror than a $20k wallet that consistently compounds on small wins.

Weight	Signal	What it catches
25	**Profitability**	Is `total_profit` meaningfully positive?
25	**Consistency**	Wins spread across tiers — no one-hit wonders
15	**Recency**	Still trading, or dormant smart money?
15	**Portfolio**	Rational mix of blue chips + alpha, or empty after rotation?
10	**Volume**	Serious wallet, or low-volume tester?
10	**Age**	Old enough to survive anti-sybil?

5.1 The One-Hit-Wonder Detector

The most distinctive part of our methodology. AVE's `smart_wallets` endpoint returns eight profit-tier buckets per wallet: `profit_above_900_percent_num`, `profit_500_900_percent_num`, `profit_300_500_percent_num`, `profit_100_300_percent_num`, `profit_10_100_percent_num`, `profit_neg10_10_percent_num`, `profit_neg50_neg10_percent_num`, `profit_neg100_neg50_percent_num`.

We define `big_wins` as any trade that returned 2x or more. Then:

```
concentration_ratio = above_900_percent_num / big_wins
```

A wallet with `concentration_ratio ≥ 0.8` had >80% of its big wins come from a single >10x bucket — classic one-hit wonder. We apply an 8-point penalty to the consistency score. This catches wallets like our live-test rank #6 (`0x6fde3 ...`): **$144,306 profit, 12% win rate**, where a single lucky trade dominated the wallet's entire P&L history.

No other public copy-trading tool computes this metric because no other data provider exposes the per-tier distribution for free.

5.2 Live Example

On 2026-04-15 against BSC with `--top 10`, AlphaMirror produced:

Rank	Score	Verdict	Total P&L	Win Rate	Portfolio	Age
1	92	**APPROVED**	+$123,518	75%	$16,931	288d
2	78	**APPROVED**	+$104,969	50%	$461,101	314d
3	68	REVIEW	+$25,606	67%	$4	238d
4	68	REVIEW	+$16,328	67%	$7,486	201d
5	64	REVIEW	+$21,255	67%	$18,210	91d
6	60	REVIEW	+$144,306	**12%**	$568,015	75d
7	55	REVIEW	+$16,648	33%	$16,941	366d
8	50	REVIEW	-$1,231	52%	$481,302	314d
9	50	REVIEW	-$14,690	52%	$461,101	314d
10	40	**REJECTED**	**-$2,344**	31%	$6,307	433d

Observations:

- Rank #6 is the one-hit wonder case. $144k profit paired with 12% win rate = one lucky trade carrying the whole wallet. Our consistency score correctly downgrades it from APPROVED to REVIEW.
- Rank #10 is the killer example. A wallet AVE has flagged as smart money for **433 consecutive days**, but the actual numbers show a **net loss of $2,344 and a 31% win rate**. AlphaMirror rejects it outright — which is exactly the product's reason to exist.

6. Technical Architecture

```
alphamirror/
├── ave_client.py       AVE REST wrapper (14 endpoints, rate limiter, auth)
├── mock_client.py      Offline fixture-backed client (DEMO_MODE=1)
├── client_factory.py   Live vs mock selection via env var
├── models.py           Typed dataclasses
│                         - Candidate (Phase 1 output)
│                         - ProfitDistribution (8-bucket histogram)
│                         - Holding (one token position)
│                         - VerifiedWallet (Phase 2 output + score)
│                         - TradeSignal (Phase 4 emit)
├── discovery.py        Phase 1 — smart_wallets fetch
├── verification.py     Phase 2 — wallet_info + wallet_tokens
├── scoring.py          6-signal weighted score
├── monitor.py          Phase 4 — poll + diff + risk gate
├── mirror.py           Phase 5 — self-custody quote builder
└── cli.py              Rich-terminal CLI (5 subcommands)

app.py                  Streamlit UI (Phases 1-5 interactive)
demo/fixtures.json      Offline sample data for DEMO_MODE=1
scripts/smoke_test.py   3-call API health check
```

6.1 Rate Limiting

AVE's free tier is **1 request per second**. AlphaMirror enforces this client-side through a thread-safe `_RateLimiter` in `ave_client.py`:

```python
class _RateLimiter:
    def __init__(self, rps: float):
        self._min_interval = 1.0 / rps
        self._last = 0.0
        self._lock = threading.Lock()

    def wait(self) -> None:
        with self._lock:
            elapsed = time.monotonic() - self._last
            if elapsed < self._min_interval:
                time.sleep(self._min_interval - elapsed)
            self._last = time.monotonic()
```

The lock matters for the Streamlit UI: multiple reruns can land concurrently, and without the lock two calls could race into a 429 error. With the lock, all requests globally serialize.

6.2 Rate-Limit Budget per Phase

Phase	Calls per run	Time on free tier
1. Discover	1	~1s
2. Verify (per wallet)	2	~2s
2. Verify (10 wallets)	20	~20s
4. Monitor (3 wallets, 1 poll)	3 (+ risk per new pos)	~4s
5. Mirror (quote)	0 (composed locally)	<0.1s

A full `run --top 10` end-to-end completes in approximately **22 seconds** on the free tier.

6.3 Why Only 2 Calls per Wallet in Phase 2

An earlier design called `address_pnl` for 5 sampled holdings per wallet to reconstruct profitability. That was **37 calls per 5-wallet run** = 37 seconds on free tier, and the data it computed was redundant with what `smart_wallets` already returned. The current design delegates profitability/win-rate computation to AVE (which already does it) and uses `wallet_info` + `wallet_tokens` purely as independent cross-checks. **7× fewer API calls, same analytical depth, strictly better data.**

7. Non-Custodial Design

AlphaMirror is non-custodial by architecture, not by marketing. The repository contains:

- **Zero private key handling code**
- **Zero seed phrase storage**
- **Zero wallet connection flows** beyond the user's local wallet app

The Phase 5 `mirror` subcommand generates a shell command like:

```
docker run --rm \
  -e AVE_API_KEY=$AVE_API_KEY \
  -e API_PLAN=free \
  --entrypoint python3 \
  ave-cloud scripts/ave_trade_rest.py quote \
  --chain bsc \
  --in-token 0x55d398326f99059fF775485246999027B3197955 \
  --out-token 0x7130d2A12B9BCbFAe4f2634d864A1Ee1Ce3Ead9c \
  --in-amount 50000000000000000000 \
  --swap-type buy
```

This command, when executed, returns an **unsigned transaction payload**. The user brings their own wallet (MetaMask, OKX, Trust, Phantom, etc.), pastes the payload, signs locally, and broadcasts. AlphaMirror sees nothing — no balance, no keys, no history beyond what's publicly on-chain.

This design choice is deliberate and aligns with the Web3 self-custody ethos that the Hong Kong Web3 Festival audience cares about. It is also a pragmatic consequence of our free-tier constraint: proxy wallet execution requires AVE API Plan Normal minimum, which requires whitelist approval.

8. How to Run

8.1 Prerequisites

- Python 3.10+ (tested on 3.13)
- AVE Cloud API key from https://cloud.ave.ai — free tier is sufficient
- Docker (optional, only needed if you want `mirror --execute` to run AVE's trade quote command inline)

8.2 Installation

```
git clone https://github.com/PugarHuda/AlphaMirror.git
cd AlphaMirror
pip install -r requirements.txt
cp .env.example .env
# edit .env and paste your AVE_API_KEY
```

8.3 CLI Usage

```
# 1. Quick sanity check — 3 API calls, verifies client works
python scripts/smoke_test.py

# 2. List raw smart-money candidates (1 API call)
python -m alphamirror.cli discover --chain bsc --top 10

# 3. Full pipeline: discover + verify + rank (end-to-end)
python -m alphamirror.cli run --chain bsc --top 5

# 4. Verify a specific wallet by address
python -m alphamirror.cli verify --wallet 0x...  --chain bsc

# 5. Build a self-custody mirror trade preview
python -m alphamirror.cli mirror --chain bsc \
    --token 0x...  --usd 50

# 6. Watch wallets for new positions (Phase 4)
python -m alphamirror.cli watch --wallet 0x...  --chain bsc \
    --interval 60 --iterations 3
```

8.4 Streamlit UI

```
python -m streamlit run app.py
```

Opens a browser at http://localhost:8501 with the interactive pipeline — sidebar controls, live Phase 1/Phase 2 progress, ranked wallet cards with score breakdowns, profit-tier distribution charts, and a per-wallet mirror trade builder.

8.5 Offline Demo Mode

For demo video recording or air-gapped development:

```
# Windows CMD
set DEMO_MODE=1
python -m alphamirror.cli run --chain bsc --top 5

# Bash / PowerShell
DEMO_MODE=1 python -m alphamirror.cli run --chain bsc --top 5
DEMO_MODE=1 python -m streamlit run app.py
```

The `MockAveClient` in `alphamirror/mock_client.py` serves canned JSON from `demo/fixtures.json` with the exact same method signatures as the live client. A 300ms simulated latency keeps visual pacing honest for video. The fixture data deliberately produces the same narrative as the live demo: 2 approved wallets, 1 rejected wallet, 1 one-hit wonder.

9. Hackathon Track Alignment

AlphaMirror targets the **Complete Application** track. It integrates:

- **Monitoring Skill** — Phase 4 (`watch`) polls tracked wallets and fires `TradeSignal` events on new positions, with automatic risk gating via the `risk` endpoint.
- **Trading Skill** — Phase 5 (`mirror`) composes self-custody trades via `trade-chain-wallet quote` , ready for the user to sign.

9.1 Judging Dimensions

Innovation (30%)

The verification-layer approach is genuinely novel. No public smart-money tool we can find — Nansen, Arkham, DeBank Smart Wallet, Telegram whale bots — uses AVE's per-tier profit distribution to detect one-hit wonders. The idea of *cross-checking* a smart-money classifier rather than either trusting it or building from scratch is also a distinct positioning.

Technical Execution (30%)

- **11 endpoints** across **2 skills** (vs. typical hackathon projects that use 3–5 endpoints from tutorials)
- **Thread-safe rate limiter** for correct behavior on free tier's 1-TPS constraint
- **Offline demo mode** with a duck-typed `MockAveClient` that mirrors the live client's exact interface — enables reliable video recording
- **Tolerant field parsers** that handle AVE's real response shape (discovered by live inspection, not guessed from docs)
- **Full type-hinted Python** with dataclass models
- **Rate-limit-aware architecture**: budgeted 2 calls/wallet in Phase 2, down from an initial 7 calls/wallet design, using `smart_wallets` data in place of reconstructed P&L

Real-World Value (40%)

- Copy-trading is a multi-billion-dollar retail crypto market

- The “which wallets should I trust” problem is universal — every retail user trying to follow alpha on Twitter runs into it
- Our rejected-wallet example (`0xacf1e ...` , 433 days as “smart money” but $2,344 net loss) is a concrete demonstration that the problem is real and currently unsolved
- The one-hit-wonder pattern (rank #6, $144k profit + 12% win rate) is a live example of a failure mode retail tools can’t detect
- Free-tier compatibility means the product is immediately accessible to every AVE Cloud user

10. Team

Team Name: Percayalah
Team Leader: Pugar Huda Mantoro
Telegram Handle: @lynx129
Email: pugarhudam@gmail.com
GitHub: https://github.com/PugarHuda

Project Repository: https://github.com/PugarHuda/AlphaMirror

Contact

For judging questions, integration discussions, or live demo requests, please reach out via the Telegram handle above or the email listed on the hackathon registration form.

11. License

MIT License — same as AVE Cloud Skill upstream, for maximum compatibility and reuse.

12. Acknowledgements

- **AVE Cloud** (https://cloud.ave.ai) for the on-chain data toolkit, free tier, and the `smart_wallets` primitive that made this verification layer possible
- **AVE Cloud Skill** (https://github.com/AveCloud/ave-cloud-skill) for the Python client source we drew the REST request shapes and config values from
- **AVE Claw Hackathon 2026** organizers for the opportunity to build on top of a well-designed skill platform

---

AlphaMirror — built for the AVE Claw Hackathon 2026. Non-custodial by design. Free-tier compatible. Deep integration by commitment.